ARCHITECTURE REVIEW & RECOMMENDATION

The 360 Platform

Service boundaries, cross-service references, single sign-on and multi-tenancy for a database-per-service SaaS ecosystem.

SCOPE

Core360 · Auth360 · Asset360 · Fleet360 · Doc360 · API Gateway

QUESTION UNDER REVIEW

How services reference data they do not own, without sharing a database

DELIVERABLE

Design discussion and one concrete target architecture. No implementation code.

DATE

21 September 2026

Contents

1 Recommendation in brief

2 Review of the submitted diagram

3 The distinction that resolves most of the questions

4 Six assumptions, judged

5 Cross-database Mongoose populate: the case against

6 Six candidate architectures compared

7 Recommended target architecture

8 Identity, single sign-on and token design

9 Multi-tenancy: is Company the tenant?

10 How tenant context flows through the system

11 API Gateway: responsibility split

12 The concrete question: rendering names Asset360 does not own

13 Failure, consistency and availability matrix

14 Decision sequencing: now, later, never

15 Open questions to resolve before building

1 Recommendation in brief

Adopt database-per-service with an event-driven backbone, and use four patterns in clearly separated roles rather than choosing one.

Pattern	Role in this platform	Applied to
Event-driven local projections PRIMARY	The backbone. Each service keeps its own read-only copy of the Core360 fields it renders.	Lists, search, filtering, sorting, reporting, detail pages, membership checks
Synchronous service API SUPPORTING	Authoritative reads where staleness is a correctness or security problem.	Write-time validation, high-risk authorization, on-demand freshness
Immutable snapshots SUPPORTING	Frozen point-in-time facts that must never change retroactively.	Invoices, signed records, completed inspections, audit trails
API composition in a BFF SUPPORTING	Client-shaped responses. Presentation concern, not a data strategy.	Screens that genuinely aggregate several services
Cross-database populate REJECT	A shared-database architecture reached through a library feature.	Nothing

On authentication: one Auth360 session produces many short-lived, audience-scoped access tokens, one per service. Single sign-on is a property of the session. Per-service authorization is a property of the tokens. The two requirements were never in conflict. CRM roles and permissions stay inside each service and never enter the token.

On tenancy: introduce an **Organization** as the tenant boundary and keep Company as a scope inside it, even if the two are one-to-one for the first two years. This costs one field today and avoids a platform-wide migration later.

> **The single highest-value decision**
>
> Each service maintains a local, event-fed projection of the Core360 fields it displays. Everything else in this document follows from that, because a field that lives in another database cannot be sorted on, filtered on or searched, no matter how good the API in front of it is.

2 Review of the submitted diagram

The diagram provided shows Core360, Fleet360, Asset360 and Doc360 all drawing arrows to a single `DB` cylinder. That is a shared-database architecture, which is precisely what the stated requirements rule out. Auth360 has no datastore of its own in the drawing, and there is no gateway anywhere.

This matters beyond the drawing. The same instinct that centralises the database also produces the question about cross-database populate. Both come from wanting a join across an ownership boundary. Most of this document is about how to get the effect of that join without the coupling it implies.

In the diagram	Consequence
One `DB` shared by four services	No isolation, no independent schema evolution, one failure domain, one release train
Auth360 reads users from Core360	Core360 availability becomes a login dependency; ownership of identity is ambiguous
Auth360 has no database	Nowhere to hold sessions, refresh tokens, signing keys or client registrations
No gateway	No single place for rate limiting, correlation IDs or edge token rejection
Direct service-to-service arrows with no contract	Coupling is implicit, so no team can see who depends on what

3 The distinction that resolves most of the questions

Separate the data needed in order to **decide** something from the data needed in order to **display** something. Almost every question in the brief has a different answer depending on which side it falls on.

Deciding	Displaying
Authorization, write-time validation, anything where being wrong is a security or integrity bug.	Names, logos, brand labels, bylines, list columns, search facets, report groupings.
Must be authoritative or cryptographically signed. Staleness is a vulnerability.	Three seconds stale is invisible to users and costs nothing.
Synchronous call, signed claim, or a projection plus a revocation list.	Local projection, always.

Worked example. "Should Asset360 call Core360 for user data?" For a permission check at write time, possibly yes. For rendering a creator name in a table of eight hundred rows, absolutely not. The question as posed has no single answer because it conflates two different problems.

4 Six assumptions, judged

"Every service needs its own MongoDB instance."

PARTLY WRONG

What is required is exclusive *ownership*: no service's code holds credentials to another service's data. That is a logical boundary, enforced with separate logical databases and separate database users carrying per-database roles. Separate clusters is a different decision, about blast radius, noisy neighbours, cost and compliance, and it is made per service when load justifies it.

Running five clusters from day one buys five sets of backups, five disaster recovery runbooks, five upgrade cycles and five monitoring configurations, and buys the architecture nothing that per-database credentials had not already bought. One exception: Auth360 should be physically isolated early, because it is the security root and its compromise and failure domains should be separate from everything else.

"The API gateway should handle all authentication."

WRONG AS STATED, RIGHT AS A FIRST PASS

The gateway rejects malformed and expired tokens early, and every service independently verifies the token anyway. The gateway is a filter, never the only line of defence. Section 11 gives the full reasoning.

"We should use Mongoose cross-database populate."

WRONG, AND WORSE THAN IT APPEARS

Section 5 makes the full case.

"We should always fetch user and company data synchronously from Core360."

WRONG FOR DISPLAY, RIGHT FOR DECISIONS

This is the distinction from section 3, and it is the axis the whole recommendation turns on.

"Company should always equal tenant."

PROBABLY WRONG, AND THE MOST EXPENSIVE ITEM ON THIS LIST

Company is the right authorization *scope*. It is a risky tenant *identity*, because "tenant" silently bundles billing, isolation, administration, data residency and lifecycle, and those come apart as soon as a customer has a group structure. Section 9 expands this.

"A single SSO token should be valid for all services."

WRONG, AND THE TENSION IS ILLUSORY

Single sign-on comes from the Auth360 session cookie. Per-service authorization comes from the `aud` claim on each short-lived access token. Both hold simultaneously. Section 8 shows the flow.

5 Cross-database Mongoose populate: the case against

The request was for a critical evaluation rather than a note that it is technically possible. Six objections, ordered roughly by severity.

1. It is not a join

MongoDB's `$lookup` cannot cross databases. Mongoose's cross-database population is an application-level fan-out of `find()` calls over a second connection. It therefore delivers the schema coupling of a shared database *and* the query cost of remote calls, with none of the query-optimiser benefit that made the idea attractive. It is the worst available combination of properties.

2. Asset360 now holds credentials to Core360's database

Isolation is not weakened, it is gone. Read-only credentials help for roughly six months, until someone needs to repair a data issue and the credential is quietly upgraded, or until a dependency vulnerability turns a read credential into an exfiltration path for every user record on the platform.

3. It couples to collection schemas, the most brittle contract available

With an API, Core360 can rename a field, change a type, introduce discriminators, split a collection or add a versioned endpoint, and consumers keep working. With populate, Core360 cannot rename `name` to `displayName` without silently breaking Asset360. There is no contract, no versioning, no deprecation window, no consumer-driven contract test, and no way for the Core360 team to discover who depends on what.

The diagnostic question: under this model, who is allowed to approve a Core360 schema migration? The answer is every team, which means nobody ships.

4. It bypasses Core360's authorization entirely

Core360's API can enforce field-level rules, tenant scoping, PII redaction and audit logging. A raw database read enforces none of it. Asset360 could read any user's email address, any company's billing fields and any soft-deleted record, leaving no trace.

5. It is invisible to operations

No HTTP spans, no per-consumer metrics, no rate limits, no circuit breakers, no timeouts under the caller's control. When Core360's database saturates, its on-call engineer sees load with no attribution. This creates a dependency the observability stack cannot see, which is a poor trade in a system where observability is a stated requirement.

6. It merges failure and deployment domains

Core360 index builds, migrations, failovers and connection-pool exhaustion all become Asset360 incidents. Asset360's test suite now needs Core360 database fixtures. The result is two services and one release train.

> **Useful mental model**
>
> Cross-service populate is a shared-database architecture that has been renamed. It is entirely fine inside one service, across collections that service owns. Across an ownership boundary it is the thing the requirements reject, arrived at through a library feature instead of a diagram.

MongoDB change streams on another service's database deserve the same verdict for the same reasons, despite feeling more modern. Subscribing to a foreign oplog makes that service's internal document shape into the event contract. Events must be published deliberately, as a versioned contract, by the owning service.

6 Six candidate architectures compared

A. Database-per-service with synchronous API calls

Asset360 calls `GET /users/{id}` and `GET /companies/{id}` whenever it needs them.

Where it works. Write-time validation, detail pages, low-volume reads. Simple, strongly consistent at read time, no new infrastructure, easy to reason about and debug.

Where it breaks, and it breaks completely. Lists. A page of five hundred assets needs a thousand calls, or two calls with batch endpoints, which should exist regardless. But batching does not clear the real wall: **sorting, filtering, searching and paginating cannot cross a service boundary.** "Assets sorted by company name", "assets where the creator's name contains a string", and "asset counts grouped by brand" each require pulling Core360's entire matching set into Asset360 or pulling every asset into memory. Both are unbounded.

Availability multiplies downward. If Core360 sits at 99.9% and every Asset360 read depends on it, Asset360 cannot exceed 99.9%. With three such dependencies the ceiling is roughly 99.7% before Asset360's own defects. Core360 becomes a single point of failure for the entire platform, which is the coupling the design set out to remove.

Consistency: strong, read-your-writes
Performance: latency adds on every request; N+1 by default
Ops complexity: lowest
Becomes problematic: at the first list view with server-side sorting
Fit: yes, as one component; no, as the strategy

B. Database-per-service with denormalized local copies, no events

Asset360 stores `companyName` on the asset at write time and never updates it.

Where it works. Point-in-time records. An invoice should state the company name as it was when issued. A maintenance record's approver should not silently change because someone edited a profile.

Where it breaks. Anything mutable. A company rename leaves thousands of stale rows with no mechanism to repair them, so backfill scripts get written by hand, indefinitely. Deleted users leave dangling references with no lifecycle.

Consistency: none; drift is permanent and unbounded
Performance: excellent, fully local
Ops complexity: low today, high in year two
Becomes problematic: at the first rename in production
Fit: yes for a deliberately frozen subset of fields; no as a general strategy

Important: B and C are different tools, not competing options. Mutable display data belongs in C. Immutable historical facts belong in B. Choose per field, not per service.

C. Database-per-service with event-driven projections

Core360 publishes `user.updated`, `company.renamed`, `membership.revoked`. Each consumer maintains a local projection collection holding only the fields it renders.

Where it works. Everything in the display category, plus lists, search, filtering, sorting and reporting, because the joinable fields now live in the same database as the query. Asset360 keeps serving reads at full speed during a Core360 outage. Adding Fleet360 or Doc360 later requires no change to Core360.

Where it costs. Eventual consistency, which must be designed for rather than hoped about. Requires a broker, an outbox in Core360 to avoid dual writes, idempotent consumers, version-ordered upserts, a dead-letter path and a reconciliation mechanism for the events that will eventually be lost. Debugging shifts from stack traces to message flows, which is a real cost to developer experience. Every new consumer adds a projection someone must own.

Consistency: eventual, typically sub-second, occasionally minutes after an incident
Performance: best reads available; single-database queries with local indexes
Ops complexity: the largest single jump, and the one worth paying
Becomes problematic: when nobody owns projector lag as an SLO
Fit: yes, this is the backbone

D. Database-per-service with CQRS and materialized views

This should not be framed as a separate option. The useful idea inside CQRS, local read models of remote data, is option C. Full CQRS means separate write and read models *inside* a service, usually with event sourcing and a command bus.

That is a large complexity bill for a CRM whose write patterns are ordinary create, read, update and delete. Take the read models. Skip event sourcing. Skip the CQRS framework. If one service later develops a genuinely different read shape from its write shape, adopt it there and only there.

Consistency: eventual, with a wider window than C
Ops complexity: high, and most of it unearned at this stage
Fit: partially, as the read-model idea already present in C

E. Database-per-service with API composition

A composition layer calls Asset360 and Core360 in parallel and stitches the results together.

Where it works. Client-shaped responses, screens that genuinely aggregate several services, keeping mobile clients from being chatty.

Where it breaks. The same wall as option A, because composition can merge results but cannot filter or sort across services. It also concentrates fan-out latency and partial-failure handling in one place, which requires timeouts, fallbacks and explicit partial-response semantics.

Placement is the real decision. Not in the gateway. In a backend-for-frontend, one per client surface, owned by the team that owns that client.

Consistency: strong per call, with partial-failure modes
Ops complexity: moderate
Becomes problematic: when it drifts into holding business rules
Fit: yes at the edge, as presentation; never as the data strategy

F. Hybrid

C as the backbone for display and query. A with batch endpoints for authoritative decisions. B for deliberately frozen historical fields. E inside a backend-for-frontend for client shaping.

This is the recommendation. The value is in the specific allocation of roles, not in the word "hybrid". A hybrid without a written rule for which pattern applies where degenerates into each developer choosing by mood.

Fit: recommended. See section 7.

Side-by-side summary

Property	A. Sync API	B. Frozen copies	C. Events	E. Composition	F. Hybrid
List of 1,000 rows	**Fails**	**Fast**	**Fast**	**Fails**	**Fast**
Sort by foreign field	**Impossible**	**Yes**	**Yes**	**Impossible**	**Yes**
Search across services	**Impossible**	Stale results	**Yes**	**Impossible**	**Yes**
Survives Core360 outage	**No**	**Yes**	**Reads yes**	**No**	**Reads yes**
Freshness	**Immediate**	**Never**	Sub-second	**Immediate**	Mixed by design
Handles renames	**Automatically**	**No**	**Yes**	**Automatically**	**Yes**
New consumer cost	Low	Low	**Replay topic**	Low	**Replay topic**
Infrastructure	**None new**	**None new**	**Broker, outbox**	BFF	**Broker, outbox, BFF**

7 Recommended target architecture

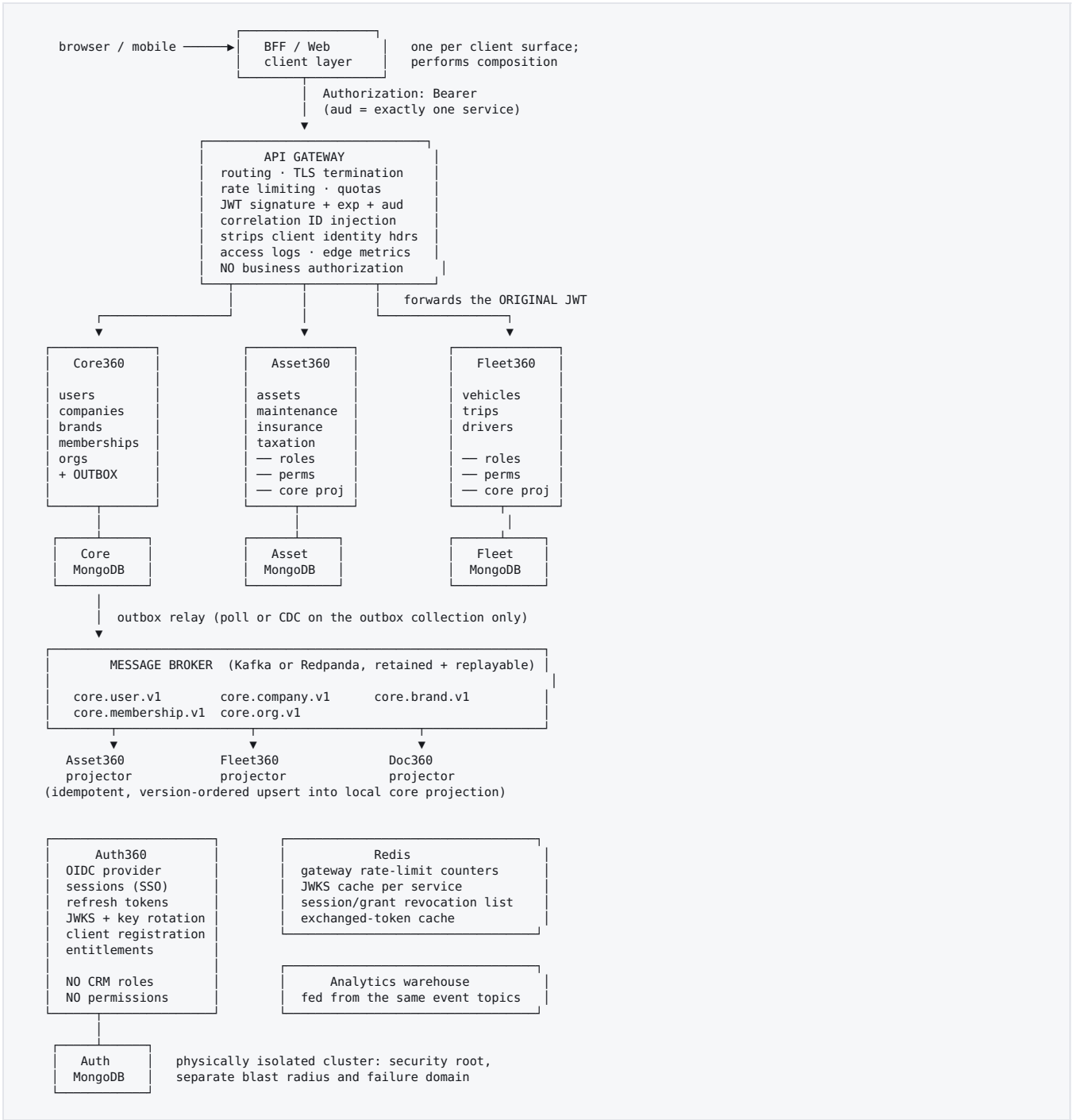


Figure 1. Target architecture. Solid ownership boundaries; no service holds credentials to another service's datastore.

Why Kafka or Redpanda rather than RabbitMQ or NATS

One specific reason: retention and replay. The brief anticipates many future 360 products. When Doc360 launches in a year, its Core360 projection should be built by replaying the topic from the beginning rather than by writing a bespoke backfill. That single property justifies the operational weight.

Pair it with a backfill API on Core360 regardless (`GET /users?updatedSince=...&cursor=...`), because replay handles new consumers while the API handles drift repair. Both will be needed.

The outbox is not optional

Core360 must write the domain change and the event row in one transaction, then have a relay publish from the outbox. Publishing directly from application code after a successful write is a dual write, and it will lose events under crash or partial

failure. The failure is silent, which is what makes it dangerous: projections drift and nobody notices until a customer reports a wrong name.



Event contract rules

- Every event carries the aggregate's monotonic **version**. Consumers upsert only when `incoming.version > local.version`. This makes them idempotent and tolerant of duplicate and out-of-order delivery.
- Topics are versioned in the name (`core.user.v1`). Additive changes go in place; breaking changes get a new topic and a dual-publish window.
- Events carry the **public contract** shape, never the internal Mongo document. Publishing the document is change-streams-by-another-name.
- Each event includes `occurredAt`, `eventId` and the originating `correlationId`, so a projection update can be traced back to the request that caused it.
- Projector lag is a first-class SLO with an alert, not an implementation detail.

Identifiers

Do not use MongoDB ObjectIds as cross-service references. Use opaque prefixed strings generated by the owning service: `usr_01H...`, `cmp_01H...`, `org_01H...`, `brd_01H...`. They survive a datastore migration, they do not leak creation timestamps, and a wrong-type identifier in a log line becomes immediately obvious. The `"COMPANY_123"` form used in the brief is already the right instinct.

8 Identity, single sign-on and token design

Who owns the user

Split this explicitly, because the submitted diagram leaves it ambiguous.

Owner	Owns	Does not own
Auth360	Credentials, MFA, sessions, the `sub` identifier, refresh tokens, signing keys, client registrations, service entitlements	Names, titles, company memberships, any CRM role
Core360	The CRM profile: display name, title, contact fields, organization and company memberships, brand associations	Passwords, sessions, tokens
Each 360 service	Its own roles, permissions and application-specific authorization	Authoritative user or company records

On first login, Core360 provisions a profile keyed by `sub`. Every service keys off `sub`, never off a Mongo ObjectId. Auth360 must not read users out of Core360's database, which would make Core360 availability a login dependency and blur authority over identity.

Resolving single sign-on against per-service authorization

One session, many audience-scoped tokens.

```
1.  User opens Asset360, is redirected to Auth360
    Authorization Code + PKCE, resource = https://api.example.com/asset360

2.  Auth360 authenticates the user once
    sets its OWN session cookie  ◀── this cookie IS the single sign-on

3.  Auth360 returns
      access token  { iss, sub, aud: "asset360", exp: +10 min,
                      scope: "asset.read asset.write", sid, jti, tenant }
      refresh token (rotating, reuse-detected)

4.  User opens Fleet360 → redirect to Auth360
    session cookie already present → NO credential prompt
    → access token { aud: "fleet360" }

5.  User opens Chat360 → Auth360 finds no client grant / no entitlement
    → DENIED at the token endpoint, before any Chat360 code executes
```

> Single sign-on comes from step 4 finding the session cookie. Per-service authorization comes from the `aud` claim in step 3 and the entitlement check in step 5. The two stated requirements were never in conflict, and no generic all-services token is needed to satisfy either.

Signing algorithm

RS256 is acceptable and has the widest library compatibility. ES256 is the better default now: signatures are a fraction of the size and verification is cheaper, which matters when every service verifies every request. Either way, include `kid` in the header, publish JWKS with overlapping key rotation, and cache JWKS in each service with a short TTL plus refresh-on-unknown-kid.

Access tokens: 5 to 15 minutes. Refresh tokens: rotating, with reuse detection. Include `sid` so an entire session can be revoked.

What belongs in the token, and what does not

In the token	Not in the token
`iss`, `sub`, `aud`, `exp`, `iat`, `jti`, `sid`, `scope`, and optionally `tenant`	CRM roles, permission lists, company membership arrays, feature flags, display names

The brief is correct to keep roles out of Auth360, and the reasons are worth recording so the decision can be defended later.

- **Unbounded growth.** Roles multiply across companies and services. A consultant serving thirty companies produces a

token too large for a header.

- **Revocation becomes impossible** within the token's lifetime. A permission removed at 10:00 still grants access until the token expires.
- **Central coupling returns.** Auth360 would become the place every service's permission model is defined, reintroducing exactly the centralisation that was removed from the database layer.

Service-to-service calls

When Asset360 calls Core360 on a user's behalf, use OAuth 2.0 Token Exchange (RFC 8693) to obtain a token with `aud: core360` that preserves `sub` as the subject and identifies Asset360 as the actor. Core360 can then enforce "Asset360 may read these profile fields, acting for this user" rather than the much weaker "Asset360 may read everything".



Cache the exchanged token for its lifetime so Auth360 is not on every hot path. For background jobs with no user present, use client credentials with a distinct audience and narrow scopes. Add mTLS or a service mesh between services so that network position alone is never sufficient authority.

The gateway should not mint its own tokens. Doing so makes it a second identity provider with its own signing keys, and downstream services lose the ability to distinguish "the user asserted this" from "the gateway asserted this".

9 Multi-tenancy: is Company the tenant?

Company is the right authorization scope for the scenarios in the brief. It is a risky tenant identity, because "tenant" silently bundles several boundaries that come apart later.

Boundary	Question it answers	Usually aligns with
Billing	Who pays, and for what plan	A group, not one company
Isolation	What a breach exposes	Organization
Administration	Who manages users and settings	Organization
Residency	Which region holds the data	Organization
Lifecycle	What is provisioned, suspended and purged together	Organization
Authorization scope	Which records a role applies to	Company

CRMs grow hierarchies: parent companies and subsidiaries, dealer groups, franchise networks, holding structures. The first customer who buys a group-wide plan and asks to see assets across all twelve of their companies breaks a model where company equals tenant. By that point the assumption is embedded in every collection, every index, every token claim and every URL.

> **Recommendation, and the cheapest high-value change available today**
>
> Introduce **Organization** as the tenant, with Company as a scope inside it. Even if the relationship is one-to-one for the first two years, a distinct `orgId` costs one field now and avoids a platform-wide migration later.
>
> If that is rejected, then at minimum keep a separate `tenantId` identifier in all APIs and tokens which happens to currently equal the company identifier, so the two can diverge without a rename across every service. Do not build the hierarchy yet. Only name the concept.

The resulting authorization model

```
Core360  (authoritative)
  Organization  org_A
    └── Company  cmp_1,  cmp_2
          └── Membership { sub: usr_1, company: cmp_1, status: active }

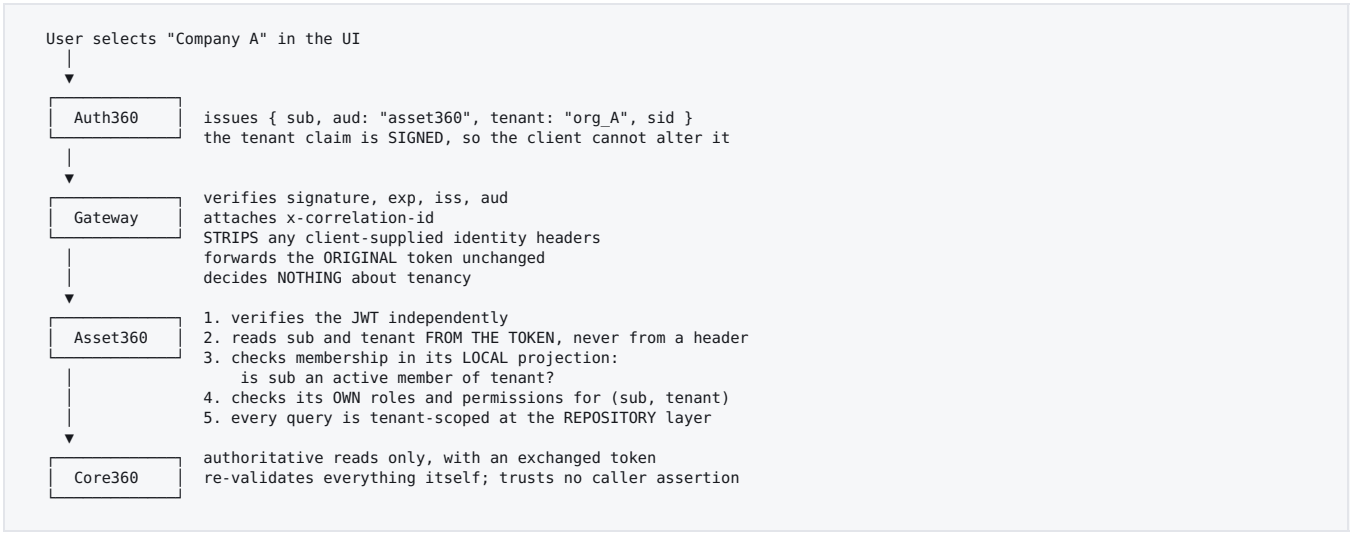
Asset360  (its own data, authoritative for its own roles)
  AssetRole { sub: usr_1, company: cmp_1, role: "asset_admin" }
  Permission set for asset_admin → asset.create, asset.delete, ...

Fleet360  (its own data, independent of Asset360)
  FleetRole { sub: usr_1, company: cmp_1, role: "fleet_viewer" }

Result: usr_1 is Asset360 Admin for cmp_1, Fleet360 Viewer for cmp_1,
        and has no Chat360 entitlement at all. Exactly as required,
        with no central role registry anywhere.
```

This confirms the model in the brief, with one change: the chain is User → Organization → Company → service-local role → service-local permissions, rather than User → Company directly.

10 How tenant context flows through the system



Answers to the specific sub-questions

Who determines the current company or tenant?

The user chooses; Auth360 signs the choice. Switching companies is a silent token refresh carrying a different `tenant` value, which takes milliseconds and is invisible to the user.

Token, header, or route?

The token, because it is the only one the client cannot forge. A route such as `/companies/{id}/assets` is fine as the request target and is useful for caching and audit, but it must then be checked for equality against the authorized tenant from the token. A header alone is never sufficient: gateways get misconfigured, internal ports get exposed, and an `X-Tenant-Id` header is a one-line privilege escalation.

Where is tenant membership validated?

In each service, against its local membership projection, on every request. This is the one place where eventually consistent data is used for an authorization decision, so be deliberate about it rather than discovering the window during an incident:

- `membership.revoked` gets priority handling on a dedicated partition or topic.
- Access token TTL stays short, so a stale grant expires quickly.
- A Redis revocation list holds recently revoked `(sub, tenant)` pairs and `sid` values; services check it on every request.
- For genuinely high-risk operations, do a synchronous authoritative check against Core360.

Accept a staleness window measured in seconds for everything else, and write that down as an explicit, agreed decision.

Where is application-specific authorization validated?

Inside each service, always, using its own roles. Asset360's roles are Asset360's data: authoritative, locally owned and instantly revocable. Never at the gateway.

How is a user prevented from changing COMPANY_A to COMPANY_B?

Layered, because any single mechanism will eventually be bypassed.

1. The `tenant` claim is signed, so tampering fails at signature verification.
2. Membership is checked server-side, so even a legitimately issued token for a company the user has since left is rejected.
3. **Tenant scoping is enforced at the repository layer, not in handlers.** One place applies the filter to every query, so no developer can forget it on a Friday afternoon.
4. Cross-tenant access returns `404`, not `403`, so existence is not leaked. Add tests that assert this.

How is cross-tenant data leakage prevented more generally?

- Compound indexes always lead with the tenant field, which makes an unscoped query slow enough to notice in staging.
- The repository layer refuses to build a query with no tenant filter, failing loudly rather than returning everything.

- Projections are tenant-scoped too. A leak in a read model is still a leak.
- Integration tests seed two tenants and assert isolation on every endpoint, as a standing requirement rather than a one-off audit.

11 API Gateway: responsibility split

Concern	Gateway	Service
Routing, TLS termination	**Yes**	**No**
Rate limiting, quotas	**Yes**	Sometimes, per tenant
JWT signature, `exp`, `iss`, `aud`	**Yes, reject early**	**Yes, again**
Correlation ID injection	**Yes**	Propagate
Access logging, edge metrics	**Yes**	Yes, domain-level
Strip client-supplied identity headers	**Yes, always**	Ignore them entirely
Response caching	Public or static only	Own caching
Tenant membership validation	**No**	**Yes**
Business authorization	**No**	**Yes**
Calling multiple services, composition	**No**	In a BFF
Caching Core360 user and company data	**No**	Via its own projection
Minting tokens	**No**	**No, Auth360 only**
Token exchange for downstream calls	**No**	**Yes, the caller does it**
Service discovery, load balancing	**Yes**	**No**

Why verify the token twice

"Validate once at the edge" sounds efficient, so the objection deserves a direct answer. Verification is a signature check against a cached key, taking tens of microseconds. It is not a performance concern at any realistic scale.

What gateway-only validation costs is that every service becomes trivially exploitable by anything reaching it off-path: service-to-service calls that do not traverse the gateway, an SSRF in any service, a misrouted path, an accidentally exposed internal port, a debug endpoint, a compromised sidecar. Zero trust means a service does not trust the network it sits on. Verify in both places.

Why composition stays out of the gateway

A gateway that holds business logic becomes a shared monolith on the critical path that every team needs to modify. That recreates the deployment bottleneck one layer up, which is the failure mode the whole architecture exists to avoid. Put composition in a backend-for-frontend owned by the client team.

12 The concrete question: rendering names Asset360 does not own

Given `Asset { companyId: "cmp_123", createdBy: "usr_123" }` in Asset360's database, and the authoritative records in Core360's, how should Asset360 display the company name, creator name and brand?

Asset360 holds `core_company` and `core_user` projection collections in its own database, containing only the fields it renders plus status. Then:

Surface	Pattern	Detail
Detail page	Projection read	Sub-millisecond, no network hop, works during a Core360 outage. If a specific page has a genuine freshness requirement, make one batch authoritative call and treat failure as "show projection values".
Lists of hundreds or thousands	Projection only	Not an optimisation. The only approach that works. The alternative is N+1 remote calls or an unbounded batch, and neither survives page 40 of a sorted table.
Search, filtering, sorting	Projection only, with real indexes	This should settle the whole debate. A field in another database cannot be sorted on or filtered by. Not with batch APIs, not with composition, not with populate. The field must be local. Everything else in this document is a consequence of that one fact.
Reporting	Projection, plus a warehouse	Projection for operational reports. For heavy analytics, publish the same events into a warehouse and report from there, so analytical load never touches service databases.
Real-time changes	Event, then upsert	Typically well under a second. If a user renames their own company and must see it immediately, either update optimistically on the client or have that single screen read through to Core360.

Lifecycle edge cases

Deleted users

Never hard-delete a referenced entity. Core360 emits `user.deactivated` or `user.deleted`; the projection sets `status: "deleted"` and **retains the display name as a tombstone**. Historical assets still read "Created by Ravi Kumar (deleted)". Assets do not cascade-delete. This is how referential integrity is achieved without foreign keys: tombstones and status flags, never dangling identifiers.

Deleted companies

Soft-delete or suspend. The projection marks the company suspended, Asset360 blocks new writes to that scope, and existing data is retained for audit and retention purposes. Genuine erasure, for example a data subject request, is an orchestrated saga: `company.purge_requested` fans out, each service purges and acknowledges, and Core360 completes only once all have. A `DELETE` on one service is not deletion in a distributed system.

Renamed companies

Two different behaviours are needed, and both must exist.

```
MUTABLE display cache              IMMUTABLE point-in-time snapshot
─────────────────────              ────────────────────────────────
core_company projection            stored ON the document at write time
follows the rename                 frozen forever
used by lists, filters, search     used by invoices, signed contracts,
                                   completed inspection records

A three-year-old invoice must NOT silently change the company name printed
on it. Decide this per field, not per service.
```

Permission changes

Asset360's own roles change instantly, because they are local and authoritative. Core360 membership revocation arrives by priority event, backed by the Redis revocation list and short token lifetimes.

Core360 outages

With projections, Asset360 serves all reads normally. Writes that require authoritative validation follow one rule:

> **Fail open for display. Fail closed for authorization.** Render a stale company name happily. Refuse to grant access on the basis of unverifiable membership.

Compare this with pure synchronous composition, where a Core360 outage takes down every product on the platform simultaneously. That is the strongest argument for projections, and the one the on-call rotation will care about most.

Projection shape

```
Asset360 database
├── assets                  (owned, authoritative)
├── asset_roles             (owned, authoritative)
├── asset_permissions       (owned, authoritative)
│
├── core_user               (PROJECTION, read-only, event-fed)
│     { _id: "usr_123", displayName, avatarUrl,
│       status: active|deleted, version, updatedAt }
│
├── core_company            (PROJECTION, read-only, event-fed)
│     { _id: "cmp_123", name, brandId, brandName, orgId,
│       status: active|suspended, version, updatedAt }
│
└── core_membership         (PROJECTION, read-only, event-fed)
      { sub, companyId, orgId, status, version }

Rules for every projection:
  · read-only to application code; only the projector writes
  · holds ONLY the fields this service renders or filters on
  · never the authoritative source for a security decision on its own
  · indexed for this service's actual queries, not Core360's
  · rebuildable from the topic at any time, and rebuilt on a schedule
```

13 Failure, consistency and availability matrix

Failure	Effect under the recommendation	Mitigation
Core360 down	All service reads unaffected. Writes needing authoritative validation are refused.	Projections; fail-closed on authorization; queue non-urgent writes
Auth360 down	New logins fail. Existing access tokens keep working until expiry.	Cached JWKS; short but non-trivial token TTL; multi-AZ Auth360
Broker down	Projections stop updating and go stale. All reads and writes continue.	Outbox retains events; projections catch up on recovery
Projector crashed or lagging	Stale display names in one service only.	Lag SLO with alerting; replay from offset; scheduled rebuild
Event lost or skipped	One record silently wrong in one projection.	Version-gated upserts; periodic reconciliation against the `updatedSince` API
Gateway down	No client traffic.	Stateless, horizontally scaled, multi-AZ; this is the one true SPOF and must be treated as such
One service's database down	That product only. Others unaffected.	The point of database-per-service
Membership revoked but token still valid	Up to one token lifetime of stale access.	Redis revocation list; priority revocation events; short TTL

Cross-service transactions

There are none, and none should be attempted. Where a workflow spans services, use a saga with explicit compensating actions and a visible state machine. Distributed transactions across MongoDB instances are both unavailable and undesirable here. Design each step to be independently retryable and idempotent, and make the intermediate states legible to support staff rather than hidden inside a framework.

14 Decision sequencing: now, later, never

Decide now	Defer safely	Never adopt
Opaque prefixed identifiers instead of ObjectIds	The broker itself. Start with batch sync APIs and a scheduled sync if necessary, but design the event contracts on day one so the migration is purely additive.	Cross-database populate
`orgId` distinct from `companyId`	Per-service MongoDB clusters	Shared collections between services
Audience-scoped tokens with no roles inside	A service mesh	Change streams on another service's database
Tenant scoping at the repository layer	Full CQRS anywhere	Roles or permissions inside the JWT
Auth360 owns `sub`; Core360 owns the profile	The organization hierarchy itself, as opposed to the identifier	Business authorization in the gateway
The outbox in Core360, from its first release		Gateway-minted tokens
Event contract shapes, even before the broker exists		Trusting an `X-Tenant-Id` header

> **One piece of pushback on the framing**
>
> The brief asks not to optimise for the easiest implementation today, which is the right instinct applied slightly too broadly. The expensive mistakes here are the ones touching identifiers, token contracts, tenant shape and data ownership, because those end up in every document and every URL. Infrastructure choices such as how many MongoDB clusters to run or which broker to pick are reversible within a sprint.
>
> Spend architectural strictness on the first category. Be pragmatic about the second.

15 Open questions to resolve before building

1. **Will any customer ever span multiple companies under one contract?** If yes, the Organization identifier is mandatory rather than advisable.
2. **What staleness window is acceptable for membership-based authorization?** This number determines token TTL, revocation-list design and whether high-risk paths need synchronous checks.
3. **Which fields on which documents must be frozen snapshots?** Produce the list per service before writing schemas, because retrofitting a snapshot is a backfill against data that no longer exists.
4. **Who owns projector lag?** If the answer is nobody, projections will drift and the event backbone will lose credibility with the team.
5. **Is there a data residency requirement on the horizon?** If yes, it constrains the tenant boundary and should be settled before section 9's recommendation is implemented.
6. **How many client surfaces will exist?** This determines whether one backend-for-frontend suffices or several are needed.

The natural next step is to specify the event contracts and projection schemas in detail, since those are the artefacts every service will depend on and the ones most expensive to change after launch.